

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 9 : Sorties Audio (Audio Output)

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

9.0 Introduction : Générer du Son

Le son, une question de fréquence

Le son n'est qu'une vibration de l'air. En faisant vibrer une membrane (haut-parleur ou buzzer) à une fréquence précise via des signaux électriques, l'Arduino peut produire des notes de musique ou des alertes sonores.

Dans ce chapitre, nous allons apprendre à :

- Connecter et utiliser un buzzer piézoélectrique.
- Utiliser la fonction `tone()` pour générer des fréquences spécifiques.
- Programmer la lecture de mélodies simples à l'aide de tableaux.
- Lire des fichiers audio numériques (WAV) depuis une carte SD.
- Communiquer avec des instruments de musique via la norme MIDI.

9.1 Le Matériel : Buzzer Piézo vs Haut-parleur

Le Transducteur Piézoélectrique :

- **Principe** : Un cristal céramique qui se déforme lorsqu'on lui applique une tension. En alternant rapidement la tension (HIGH/LOW), il crée une onde sonore.
- **Avantage** : Consomme très peu de courant, peut être branché **directement** sur une broche de l'Arduino.

Le Haut-parleur classique (Magnétique) :

- **Principe** : Une bobine mobile dans un aimant.
- **Inconvénient** : Consomme trop de courant (impédance de 8Ω). Il nécessite un circuit d'amplification (transistor ou LM386) ou une résistance série en dépannage pour ne pas griller l'Arduino.

[Insérer le schéma de câblage d'un buzzer piézo (GND et broche 8)]

9.2 Jouer une note avec tone()

La fonction `tone(broche, frequence, duree)` Génère un signal carré (onde) à la fréquence spécifiée (en Hertz) sur la broche choisie.

- Le La central (A4) correspond à 440 Hz. Le Do moyen (C4) correspond à environ 261 Hz.
- L'appel à `tone()` ne bloque pas l'exécution du code (fonctionne en tâche de fond grâce aux timers matériels de l'Arduino).

```
const int buzzerPin = 8;

void setup() {
  // Joue un 'La' (440 Hz) pendant 1000 millisecondes (1 seconde)
  tone(buzzerPin, 440, 1000);

  // Attendre la fin de la note car tone() est asynchrone
  delay(1000);
}

void loop() {
  // Vide
}
```

9.3 Limites et spécificités de tone()

Limitations matérielles

La fonction tone() utilise un Timer interne de l'Arduino (le Timer 2 sur l'Uno).

- L'utilisation de tone() perturbe la sortie PWM sur les broches 3 et 11.
- **Règle d'or** : L'Arduino ne peut jouer qu'une seule note à la fois.

Arrêter le son manuellement : Si vous n'avez pas spécifié de durée dans tone(), le son continuera indéfiniment. Il faut utiliser noTone(broche).

```
void loop() {  
    tone(8, 440); // Demarre le son  
    delay(500);   // Laisse sonner 0.5s  
    noTone(8);    // Coupe le son  
    delay(500);   // Silence de 0.5s  
}
```

9.4 Jouer une mélodie complète

Problème : Comment jouer une chanson complexe sans écrire des centaines de lignes `tone()` et `delay()` ?

Solution : Utiliser des tableaux (Arrays) pour stocker la partition.

Il nous faut deux tableaux distincts (ou un tableau de structures) :

1. **Mélodie :** Un tableau contenant les fréquences des notes.
2. **Rythme :** Un tableau contenant la durée relative de chaque note (Noire = 4, Croche = 8, Blanche = 2).

Astuce : Le fichier d'exemple `pitches.h` fourni dans l'IDE Arduino contient des constantes lisibles pour toutes les notes (ex : `NOTE_C4` pour le Do, `NOTE_A4` pour le La).

Code : Algorithme de lecture de mélodie

```
#include "pitches.h" // A inclure pour utiliser NOTE_C4, NOTE_G3, etc.

const int buzzerPin = 8;
// Partition : Do, Re, Mi, Fa, Sol, La, Si, Do
int melodie[] = {NOTE_C4, NOTE_D4, NOTE_E4, NOTE_F4, NOTE_G4, NOTE_A4, NOTE_B4, NOTE_C5};
int dureeNotes[] = {4, 4, 4, 4, 4, 4, 4, 4}; // 4 = Noire

void setup() {
    for (int i = 0; i < 8; i++) {
        // Calculer la duree en millisecondes (1 seconde / type de note)
        int dureeReelle = 1000 / dureeNotes[i];
        tone(buzzerPin, melodie[i], dureeReelle);

        // Pause entre les notes (130% de la duree de la note pour un effet staccato)
        delay(dureeReelle * 1.30);
        noTone(buzzerPin);
    }
}

void loop() {}
```

9.5 La synthèse sonore complexe

La fonction `tone()` produit uniquement une onde carrée parfaite, ce qui donne un son très électronique et agressif (façon "jeux vidéo 8-bits").

Pour créer des sons plus doux ou des bruitages complexes (sirènes, sons de lasers), nous devons faire varier rapidement la fréquence (balayage / sweep) dans une boucle `for`.

[Insérer l'image d'une forme d'onde carrée]

Code : Générer une sirène d'alarme

```
const int buzzerPin = 8;

void loop() {
    // Mont e en fr quence (Aigu)
    for (int freq = 500; freq <= 1000; freq += 10) {
        tone(buzzerPin, freq);
        delay(10);
    }

    // Descente en fr quence (Grave)
    for (int freq = 1000; freq >= 500; freq -= 10) {
        tone(buzzerPin, freq);
        delay(10);
    }
}
```

9.6 Jouer de vrais fichiers audio (WAV)

Problème : L'Arduino n'a ni la mémoire (2 Ko de RAM) ni la puissance pour décoder un fichier MP3 (qui est très compressé).

Solution : Lire un fichier audio non compressé (Format WAV, PCM 8-bits, Mono, 16kHz max) stocké sur une **carte SD**.

Matériel requis :

- Un module lecteur de carte MicroSD (communiquant via le bus SPI : broches 11, 12, 13 et CS).
- Un haut-parleur (et son circuit d'amplification) connecté sur une broche PWM (ex : broche 9).

[Insérer l'image d'un module MicroSD connecté à l'Arduino]

9.7 La bibliothèque TMRpcm

Pour lire du son directement avec l'Arduino (sans puce de décodage MP3 externe type DFPlayer), la bibliothèque TMRpcm est la plus utilisée. Elle manipule les timers très rapidement pour convertir les données du fichier en signal analogique via le PWM (Digital to Analog Conversion - DAC).

```
#include <SD.h>           // Pour lire la carte SD
#include <TMRpcm.h>        // Pour lire l'audio
#include <SPI.h>

TMRpcm audio; // Créer l'objet audio
const int sdChipSelect = 4;

void setup() {
  audio.speakerPin = 9; // DOIT etre la broche 9 (ou 10) sur Uno !
  Serial.begin(9600);

  if (!SD.begin(sdChipSelect)) {
    Serial.println("Erreur Carte SD !");
    return;
  }
  audio.play("musique.wav"); // Lancer la lecture
}
```

9.8 Qu'est-ce que le protocole MIDI ?

MIDI (Musical Instrument Digital Interface) : Créé dans les années 80, le MIDI ne transmet **aucun son**. Il transmet des **données de performance** (des instructions).

Exemple d'instructions MIDI :

- "Appuie sur la touche Do moyen avec une force de 80." (Note On)
- "Relâche la touche." (Note Off)
- "Change l'instrument pour une guitare." (Program Change)

L'intérêt pour l'Arduino : Il est extrêmement facile pour l'Arduino de générer ces messages textes via sa broche série (TX) et de piloter des synthétiseurs professionnels ou des logiciels MAO (Ableton, FL Studio).

9.9 Câblage d'une sortie MIDI

Le câble MIDI standard utilise un connecteur DIN à 5 broches.

- La transmission se fait via une boucle de courant (nécessite des résistances de $220\ \Omega$).
- Vitesse de communication standard très spécifique : **31250 bauds**.

[Insérer le schéma de câblage d'une prise DIN 5 broches (Broche TX de l'Arduino vers DIN broche 5)]

Code : Envoyer des messages MIDI

Un message Note On MIDI est composé de 3 octets (bytes) : 1. Le statut (Type de message et canal). Exemple : 0x90 = Note On, Canal 1. 2. La note (0 à 127). Exemple : 0x3C = 60 (Do moyen). 3. La vélocité (Force de frappe, 0 à 127).

```
void setup() {  
    // Vitesse de communication obligatoire pour la norme MIDI  
    Serial.begin(31250);  
}  
  
void loop() {  
    // Joue la note 60 (Do moyen)  
    envoyerMIDI(0x90, 60, 100);  
    delay(1000);  
  
    // Arrête la note 60 (Vélocité à 0 = Note Off)  
    envoyerMIDI(0x90, 60, 0);  
    delay(1000);  
}  
  
void envoyerMIDI(byte commande, byte donnee1, byte donnee2) {  
    Serial.write(commande);  
    Serial.write(donnee1);  
    Serial.write(donnee2);  
}
```

9.10 Synthétiseurs Matériels Dédiés

Pour éviter de surcharger l'Arduino avec la création d'ondes sonores, et obtenir une qualité "CD" (MP3), on utilise très souvent des composants dédiés (Breakout Boards).

Exemple : Le DFPlayer Mini

- Contient son propre lecteur de carte SD.
- Décode matériellement le format MP3.
- Intègre un petit amplificateur audio (3W) pour piloter directement un haut-parleur.
- L'Arduino lui envoie juste des ordres série très simples : " Joue le fichier 3", "Volume 15", "Pause".

Résumé du Chapitre 9

- **La fonction `tone()`** : Parfaite pour des bips et des mélodies simples "8-bits". Ne gère qu'un seul son à la fois.
- **Matériel sonore** : Protégez vos broches. Un buzzer piézo se branche directement, un véritable haut-parleur magnétique nécessite une amplification ou une résistance série de protection.
- **Audio complexe (WAV)** : Possible via la bibliothèque `TMRpcm` et une carte SD, mais la qualité reste basique (8-bits, PWM). Privilégiez un module dédié (type `DFPlayer`) pour les voix ou la vraie musique.
- **MIDI** : L'Arduino est l'outil parfait pour créer des contrôleurs musicaux sur-mesure (claviers, batteries électroniques) en envoyant des instructions standardisées à des ordinateurs ou des synthétiseurs.

Fin du Chapitre 9. Le Chapitre 10 abordera le contrôle à distance des appareils externes (Infrarouge, Télécommandes, Relais).